SOC LAB REPORT

# File Integrity Monitoring with Wazuh

Detecting file creation, modification and deletion on a Windows 11 endpoint using a Wazuh Manager and Agent

| Item | Detail |
|---|---|
| Student name | Sikiru Habeeb |
| Registration No. | C11/26/EHIT/17330 |
| Course / Lab | CIPA107 \| Wazuh SOC Lab: File Integrity Monitoring |
| Report date | October 20, 2026 |
| Wazuh version | 4.12.0 (manager and agent) |
| Manager | icdfa-server (Ubuntu, VirtualBox, bridged adapter) |
| Agent | Windows-soc-lab, ID 004 (Windows 11, VirtualBox) |

# 1. Introduction and Objectives

File Integrity Monitoring (FIM) detects unauthorized or unexpected changes to files by comparing their current state (hashes, size, permissions, timestamps) with a known baseline. It is a core detective control for catching tampering, malware persistence and ransomware activity, and it supports compliance requirements such as PCI DSS 11.5.

In this lab, a Wazuh Manager running on an Ubuntu virtual machine was paired with a Wazuh Agent on a Windows 11 virtual machine. The agent was configured to watch a test folder in real time, and the resulting alerts were investigated from the Wazuh Dashboard as a SOC analyst would.

**Objectives**

- Access the Wazuh Dashboard of the manager and confirm the platform is healthy.
- Install the Wazuh Agent on Windows and register it with the manager using a unique authentication key.
- Enable real-time FIM on a monitored folder (SensitiveFiles).
- Generate create, modify and delete events and investigate the alerts in the dashboard.
- Answer the four analysis questions on alert triage, false positives, ransomware detection and tool comparison.

# 2. Lab Architecture and Environment

| Component | Host | Role |
|---|---|---|
| Wazuh Manager | Ubuntu Server VM (icdfa-server), VirtualBox, bridged adapter | Collects, analyzes and stores data from agents; hosts the indexer and dashboard. |
| Wazuh Agent | Windows 11 VM (user Windows11), VirtualBox | Endpoint sensor; runs the syscheck (FIM) module and ships events to the manager. |
| Dashboard | Browser on the Windows VM | SOC console used to monitor agents and investigate alerts. |

**Scope note.** Part 1 of the lab (installing the Wazuh Manager) was skipped because a preconfigured Wazuh Ubuntu server was provided. The manager, indexer and dashboard services were simply started at the beginning of each session and stopped at the end.

**Addressing.** Because the manager uses a bridged adapter with DHCP, its address differed between sessions: 10.53.20.149 on October 18, 2026 and 192.168.0.179 on October 20, 2026. The Windows agent reported the address 192.168.0.108 in the alerts. Dashboard URLs in the screenshots reflect these addresses.

# 3. Procedure and Evidence

## 3.1 Starting the manager and accessing the dashboard (Part 2)

On the Ubuntu server the three Wazuh services were started with systemctl (manager, indexer, dashboard). The interface address was confirmed with ip addr, which showed 10.53.20.149 on enp0s3, a bridged address on the same network as the Windows VM.

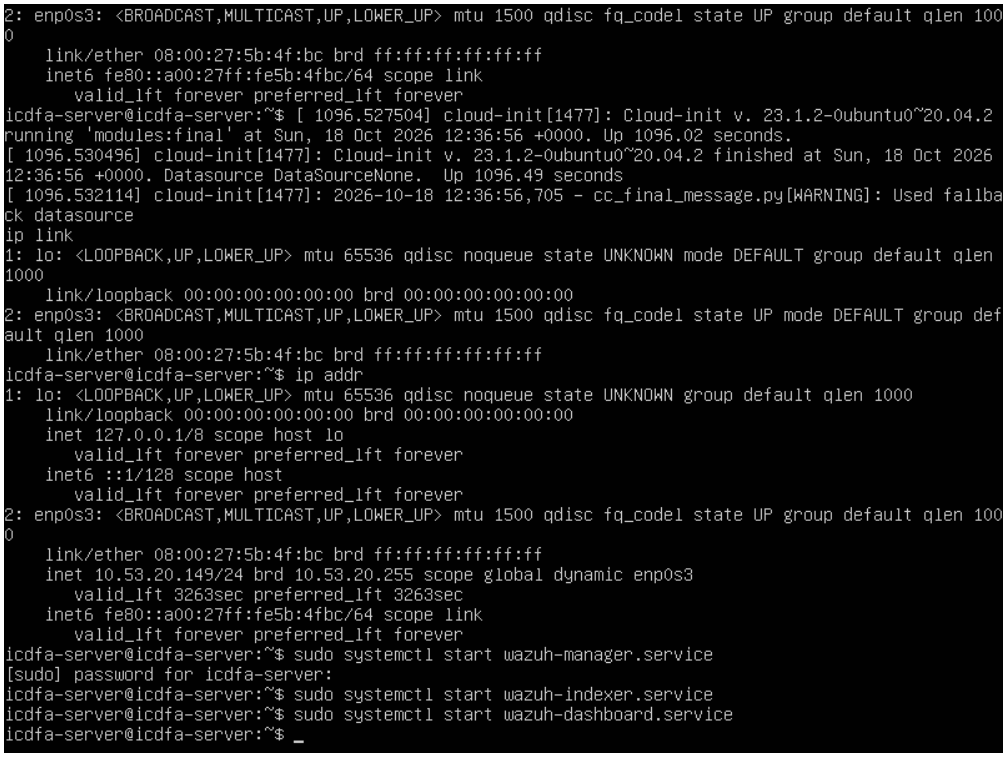



***Figure 1:*** *Wazuh manager, indexer and dashboard services started on icdfa-server; ip addr shows 10.53.20.149 on enp0s3.*

From the Windows VM, the dashboard was opened in a browser at https://10.53.20.149 and the self-signed certificate warning was accepted, as expected in a lab. The dashboard loaded and the Overview page was reached after logging in.

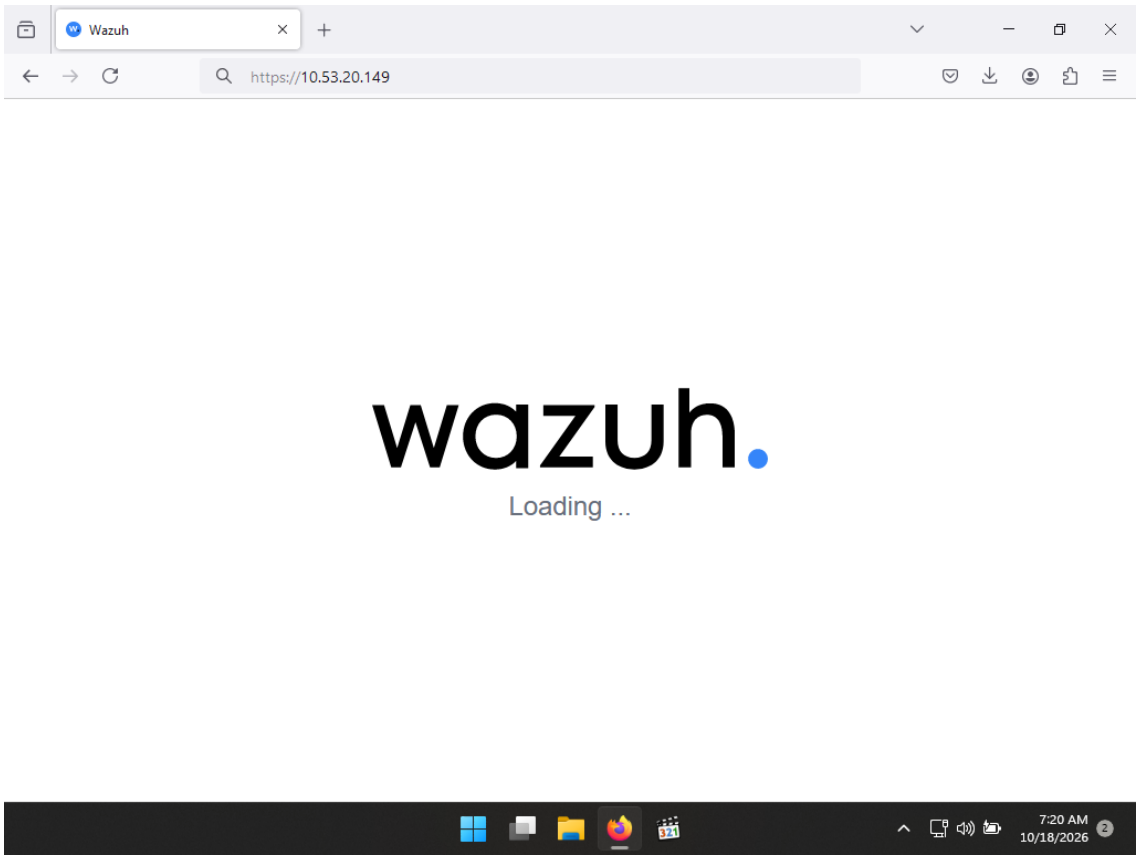



***Figure 2:*** *Wazuh Dashboard loading at https://10.53.20.149 (October 18, 2026, 7:20 AM).*

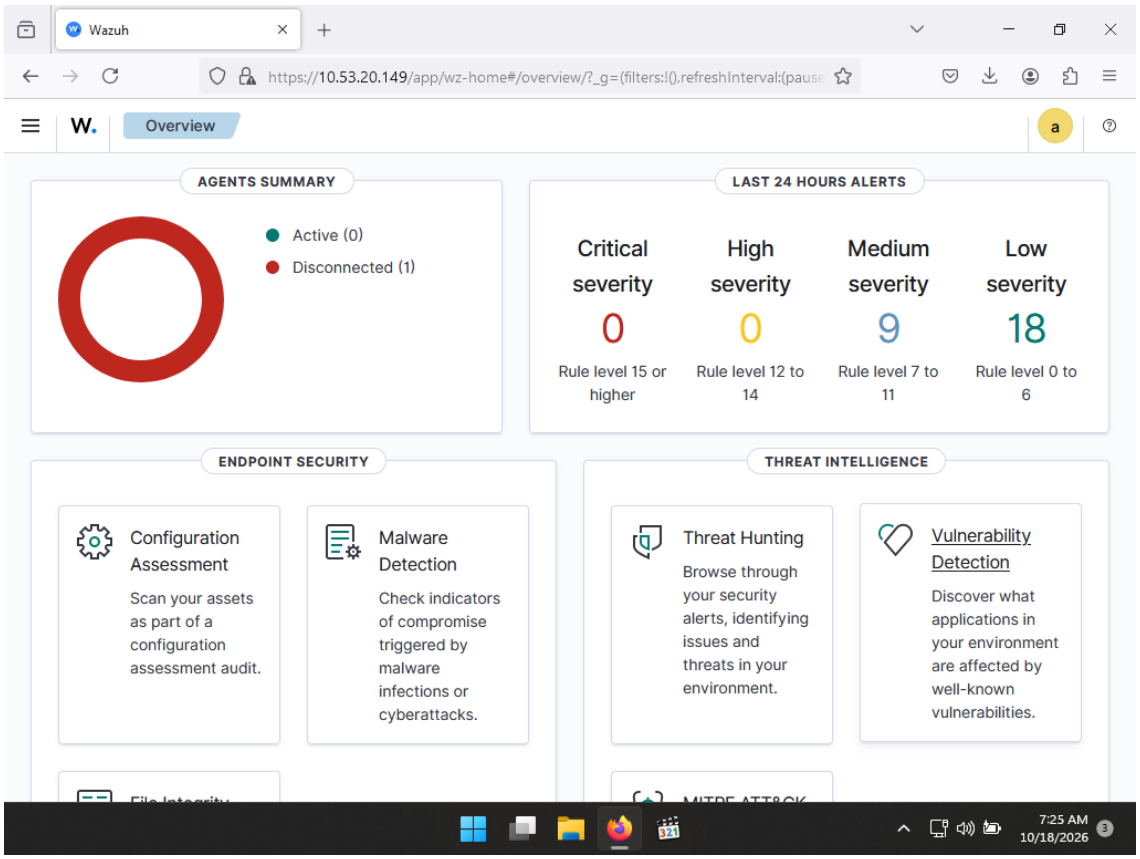



***Figure 3:*** *Wazuh Overview page before the new agent was registered: Active (0), Disconnected (1) (7:25 AM).*

## 3.2 Installing the Wazuh Agent (Part 3)

The Wazuh Agent 4.12.0 MSI installer was downloaded from the official Wazuh package site and run with the default settings. The installer reached the final stage of the installation progress bar.

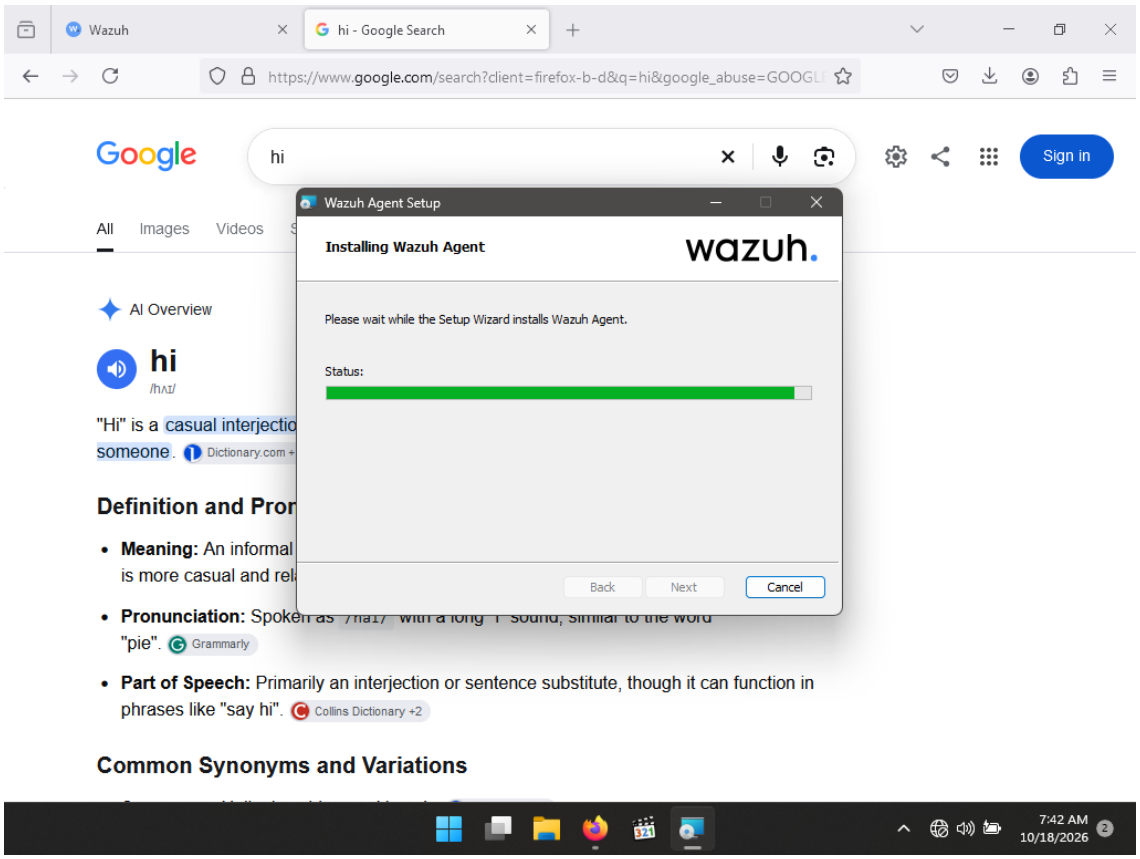



***Figure 4:*** *Wazuh Agent Setup wizard installing the agent with default settings (October 18, 2026, 7:42 AM).*

## 3.3 Agent-manager registration (Part 4)

On the manager, `/var/ossec/bin/manage_agents` was run with sudo. A new agent named **Windows-soc-lab** with the IP address **any** was added and received **ID 004**. Its key was extracted with option E and pasted, together with the manager IP address, into the Wazuh Agent Manager on Windows, after which the agent service was restarted.

An earlier registration attempt on the server is also captured. The first command was mistyped (manage-agents, command not found) and then re-run correctly; it created an agent named windows_SOC_LAB1 (IP 10.0.2.15) with ID 003, and its key was extracted. The final agent used in this lab is ID 004, Windows-soc-lab.

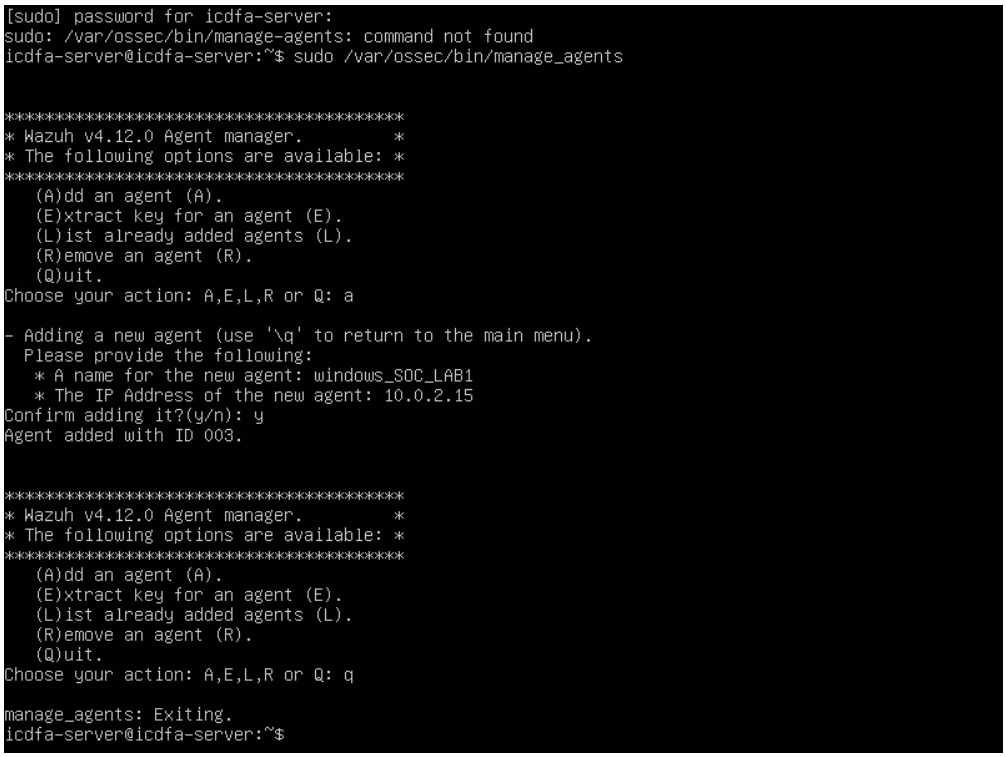



***Figure 5:*** *manage_agents on the server: mistyped command (command not found), then agent windows_SOC_LAB1 (10.0.2.15) added with ID 003.*

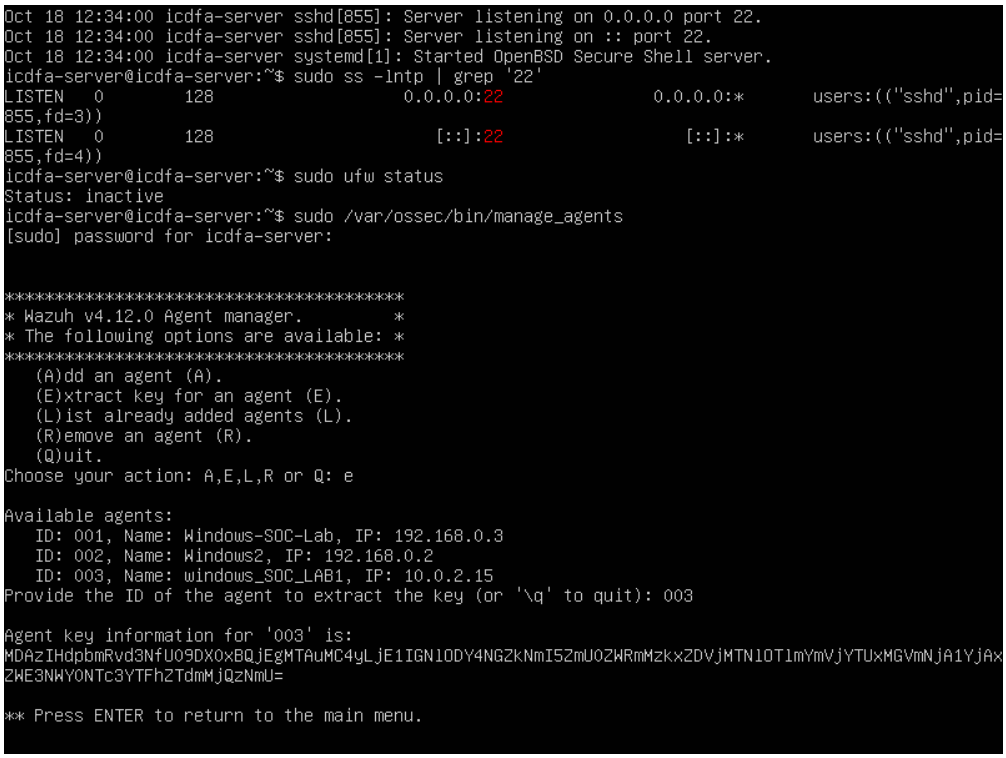



***Figure 6:*** *sshd and ufw checks, then manage_agents option E listing agents 001, 002 and 003 and extracting the key for 003.*

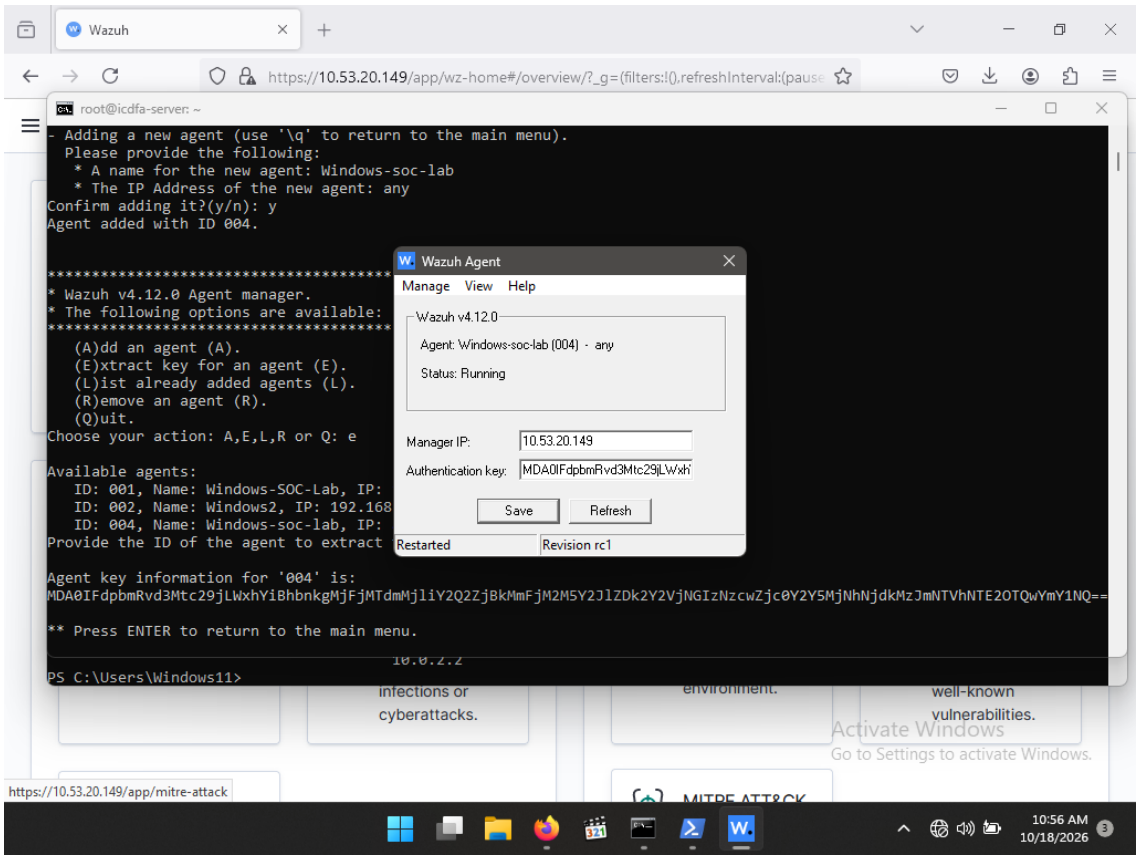



***Figure 7:*** *Final registration: agent Windows-soc-lab added with ID 004 (IP any), key extracted, and the Wazuh Agent Manager showing the manager IP, key and Status: Running (October 18, 2026, 10:56 AM).*

The Endpoints page of the dashboard confirmed the registration: the agent counts as Active. The other two entries (Disconnected and Never connected) are agent records that already existed on the preconfigured server and are not part of this lab.

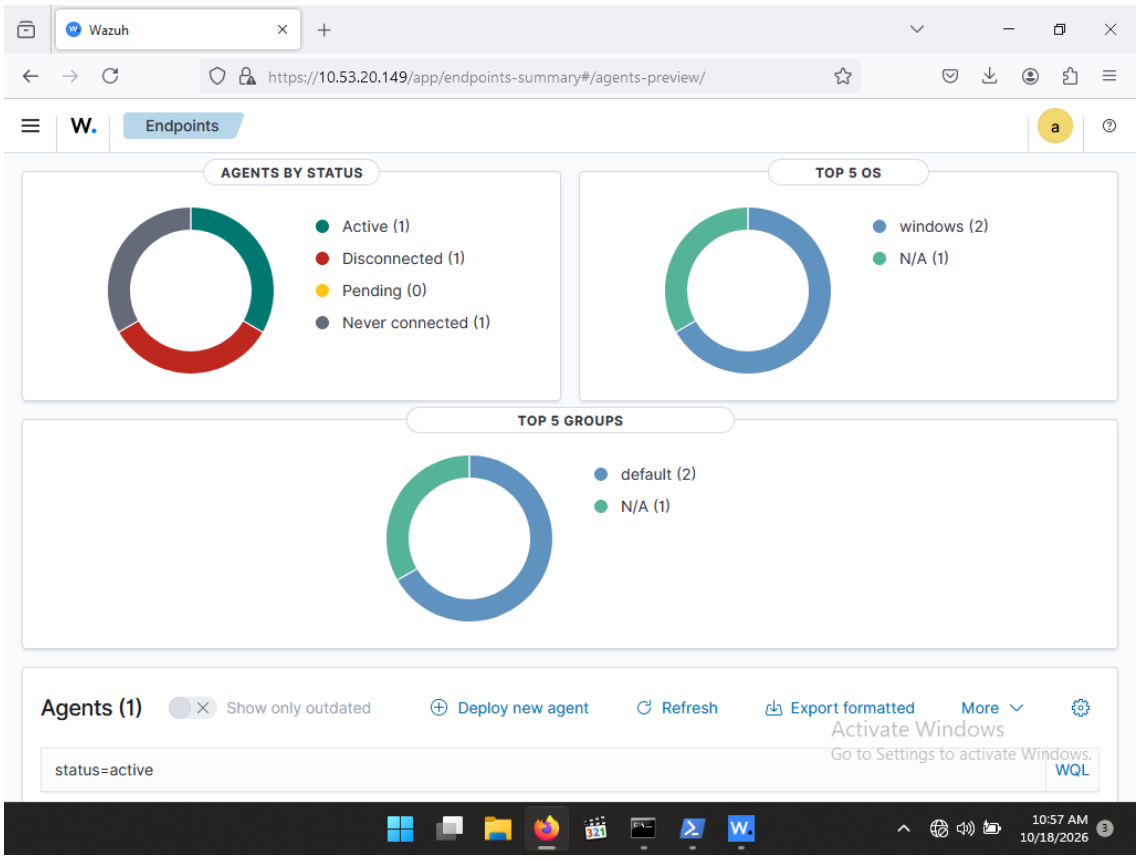



***Figure 8:*** *Endpoints page showing Active (1) after the agent key was applied (October 18, 2026, 10:57 AM).*

## 3.4 Enabling real-time FIM (Part 5)

The syscheck section of the agent configuration file `C:\Program Files (x86)\ossec-agent\ossec.conf` was edited to add a real-time directory entry for the test folder, and the file was saved. The alerts later received from the manager report the monitored path as `c:\users\windows11\sensitivefiles`.

```
<directories realtime="yes">C:\Users\<username>\SensitiveFiles</directories>
```

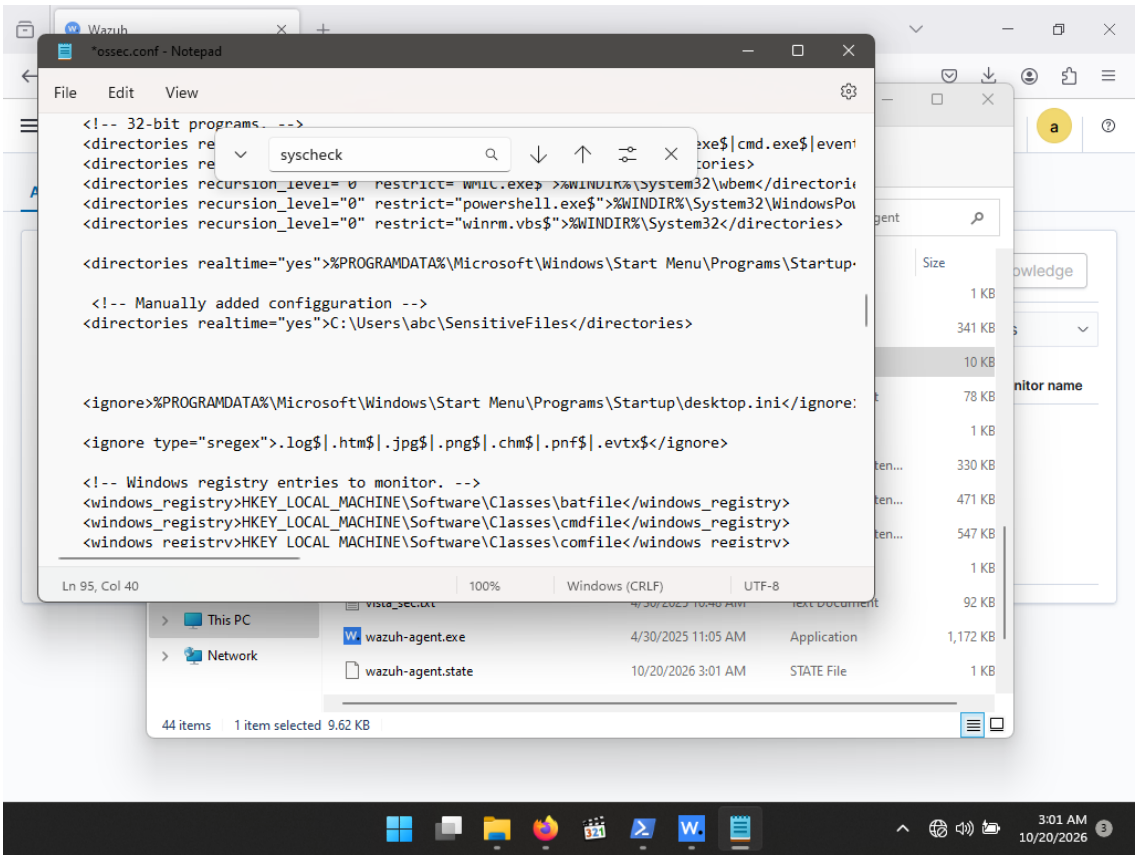



***Figure 9:*** *ossec.conf open in Notepad, searching for syscheck; the manually added realtime directories line for SensitiveFiles is visible (October 20, 2026, 3:01 AM). In this view the entry reads C:\Users\abc\SensitiveFiles, the lab template path.*

The agent was then restarted from the Wazuh Agent Manager (Manage > Restart) so that the new configuration took effect. The status bar shows Restarted.

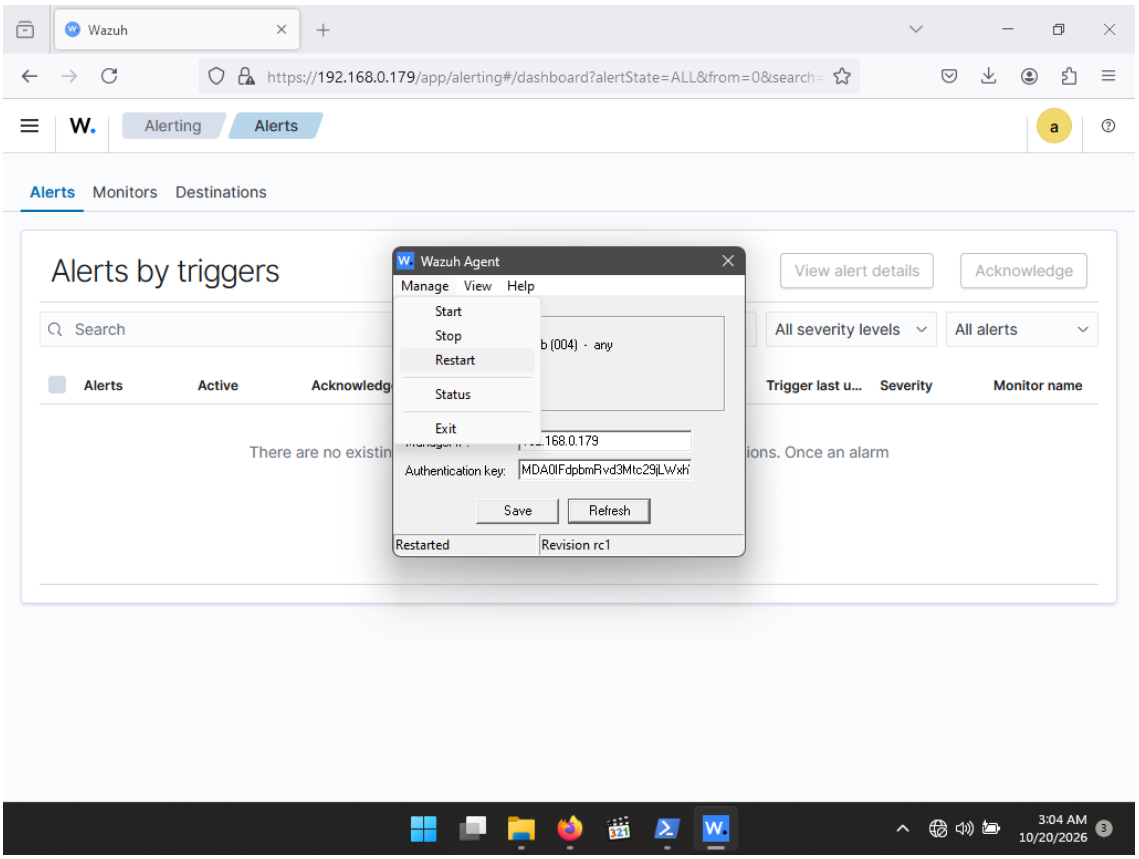



***Figure 10:*** *Wazuh Agent Manager: Manage > Restart applied after the configuration change (October 20, 2026, 3:04 AM).*

## 3.5 Generating test events (Part 6)

The folder `C:\Users\Windows11\SensitiveFiles` was created and the three required actions were performed: a text file was created and renamed to **secret_plan.txt**, text was added and saved, and the file was then deleted.

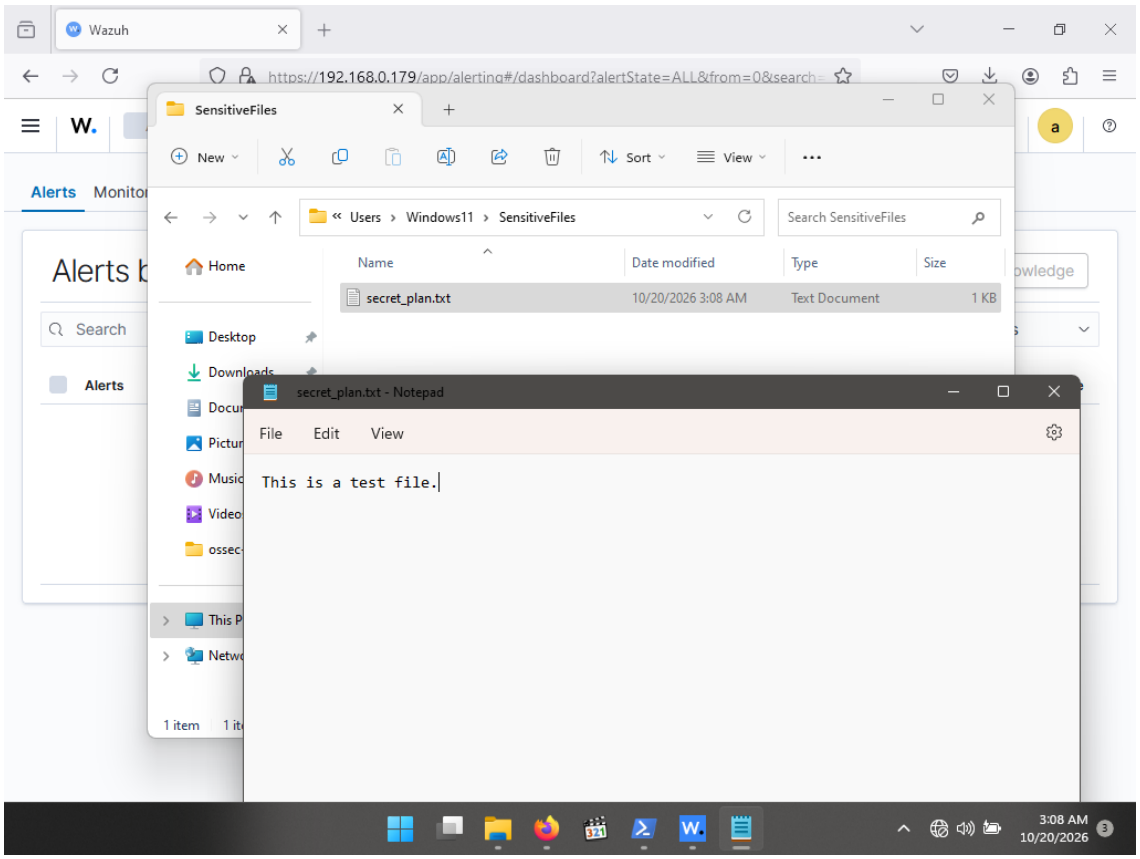



***Figure 11:*** *Earlier test: secret_plan.txt in SensitiveFiles containing "This is a test file." (October 20, 2026, 3:08 AM). The run analysed in Section 4 used different content.*

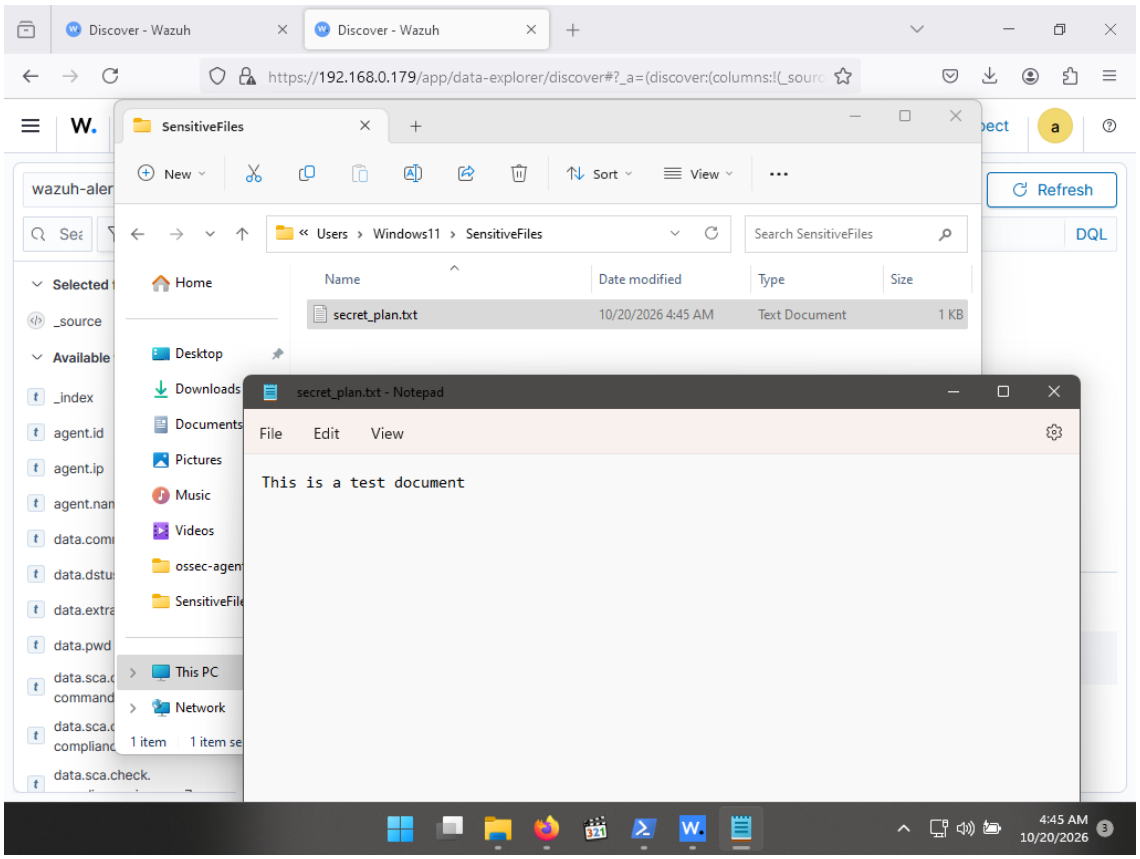



***Figure 12:*** *secret_plan.txt in SensitiveFiles after the text was added and saved (4:45 AM). The Discover view with the FIM alerts is open behind it.*

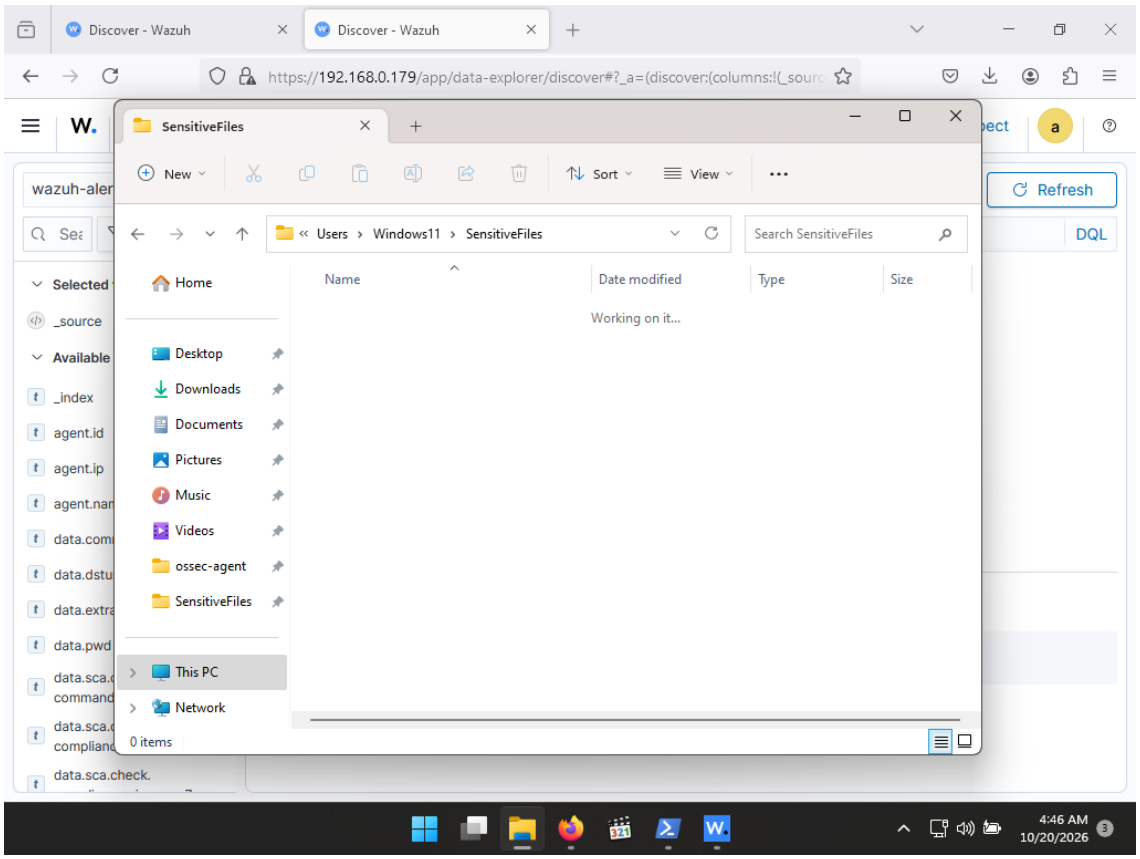



***Figure 13:*** *SensitiveFiles after secret_plan.txt was deleted; the folder now contains 0 items (4:46 AM).*

## 4. Alert Investigation

In the dashboard, the Discover view was opened on the wazuh-alerts-* index pattern with the filter rule.groups: syscheck, which limits the results to File Integrity alerts. The filter returned five hits for the last 24 hours, all arriving within roughly two minutes of the test activity.

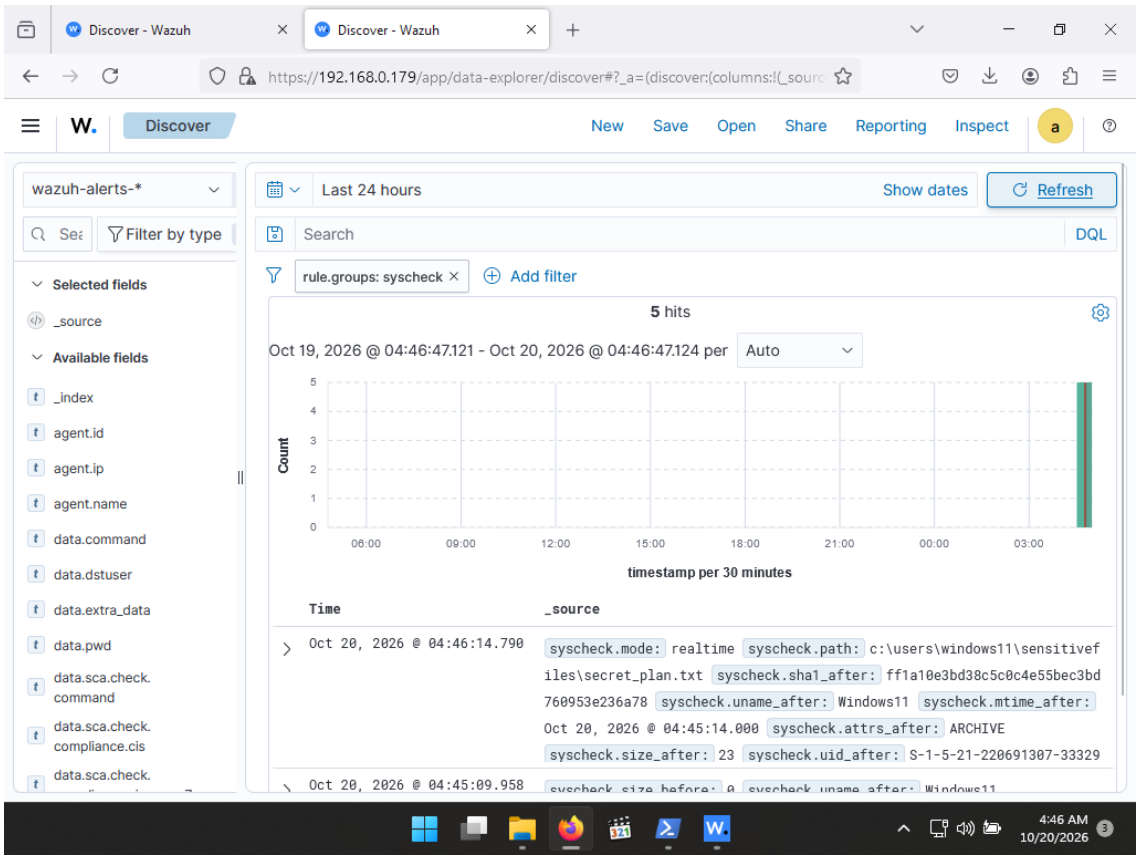



***Figure 14:*** *Discover with filter rule.groups: syscheck showing 5 hits; the newest alert is for secret_plan.txt at Oct 20, 2026 @ 04:46:14.790.*

The five alerts, in chronological order, tell the complete story of the test:

| Timestamp | Event | File | Result |
|---|---|---|---|
| Oct 20, 2026 @ 04:44:42.117 | added | new text document.txt | New file |
| Oct 20, 2026 @ 04:44:48.839 | deleted | new text document.txt | Renamed away |
| Oct 20, 2026 @ 04:44:48.863 | added | secret_plan.txt | New name |
| Oct 20, 2026 @ 04:45:09.958 | modified | secret_plan.txt | Content added |
| Oct 20, 2026 @ 04:46:14.790 | deleted | secret_plan.txt | File removed |

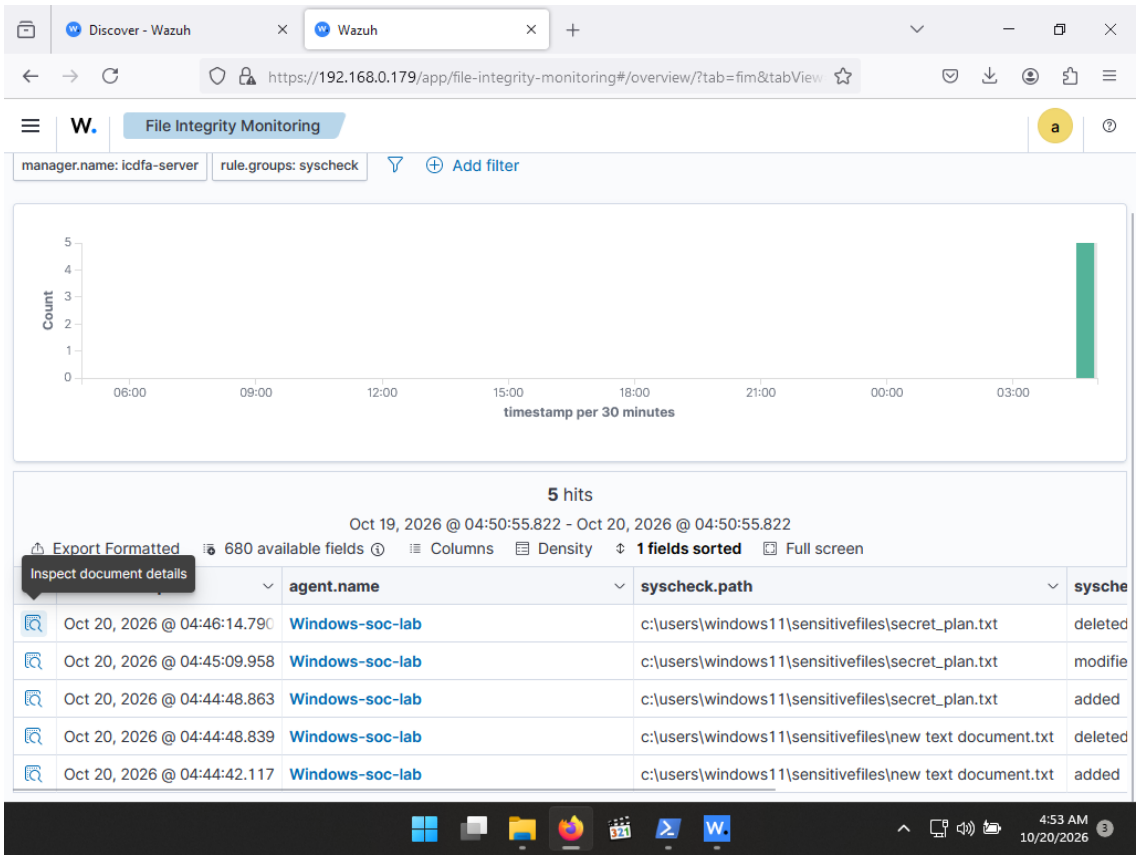



***Figure 15:*** *File Integrity Monitoring view listing the five syscheck events: added, deleted, added, modified and deleted (October 20, 2026, 4:53 AM).*

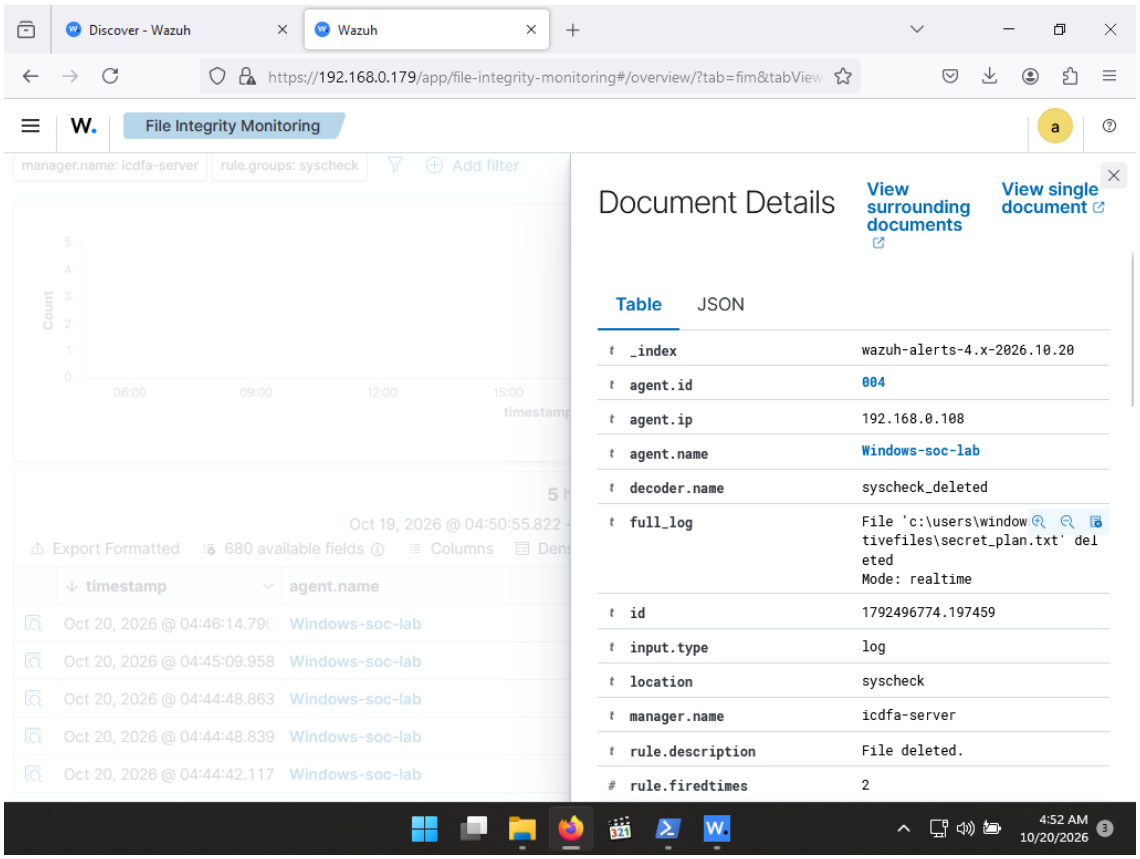



***Figure 16:*** *Document details of the deleted alert for secret_plan.txt: decoder syscheck_deleted, rule description "File deleted.", mode realtime (4:52 AM).*

The delete and add pair at 04:44:48 is consistent with the file being renamed from the Windows default name (New Text Document.txt) to secret_plan.txt, which syscheck reports as the removal of one path and the creation of another. All events carry mode: realtime, meaning they were reported as they happened rather than at the next scheduled scan.

## 5. Lab Challenge and Analysis Questions

### Question 1: Alert Triage

**Task:** Find an alert where a file was modified. What specific details in the alert tell you what was changed about the file?

The modified-file alert is the one raised at Oct 20, 2026 @ 04:45:09.958 for c:\users\windows11\sensitivefiles\secret_plan.txt. It was generated by rule 550, "Integrity checksum changed" (level 7), decoded by syscheck_integrity_changed, and sent by agent 004 (Windows-soc-lab) to manager icdfa-server.

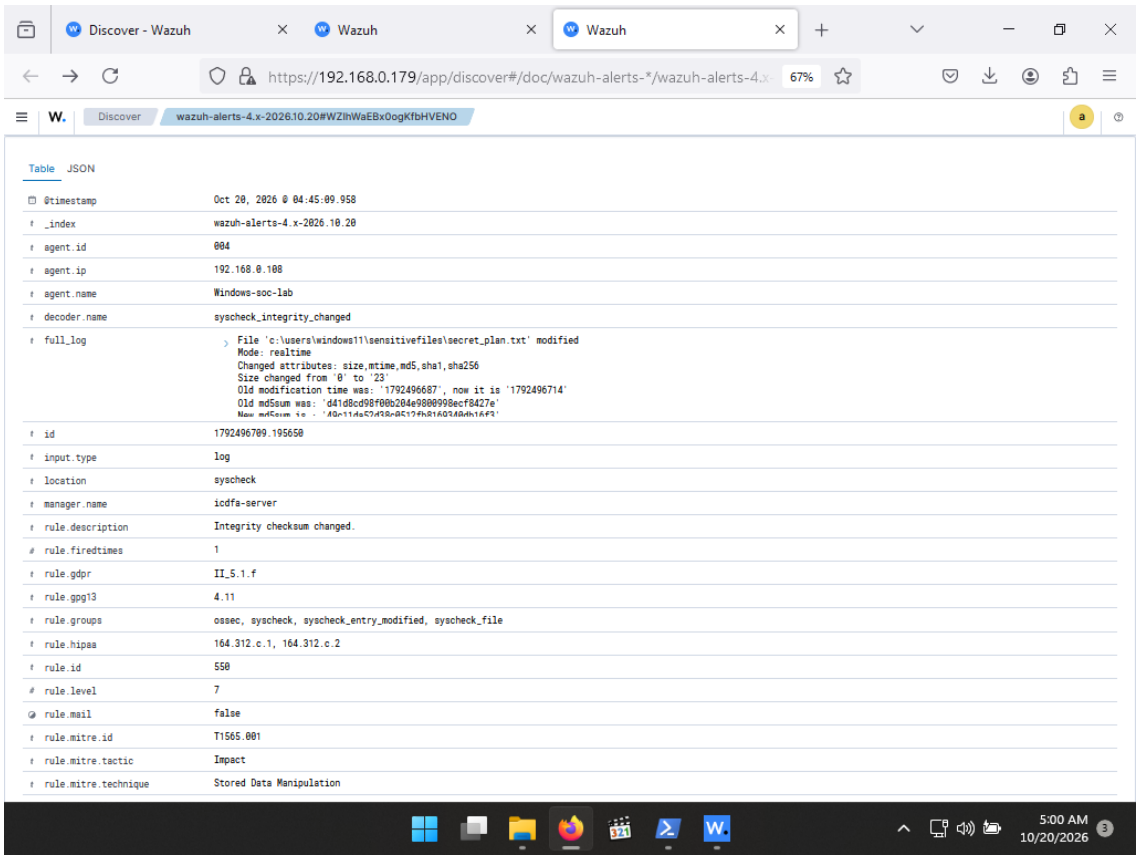



***Figure 17:*** *Document details of the modified alert: decoder, full_log, rule description, rule groups and MITRE mapping (top of the document).*

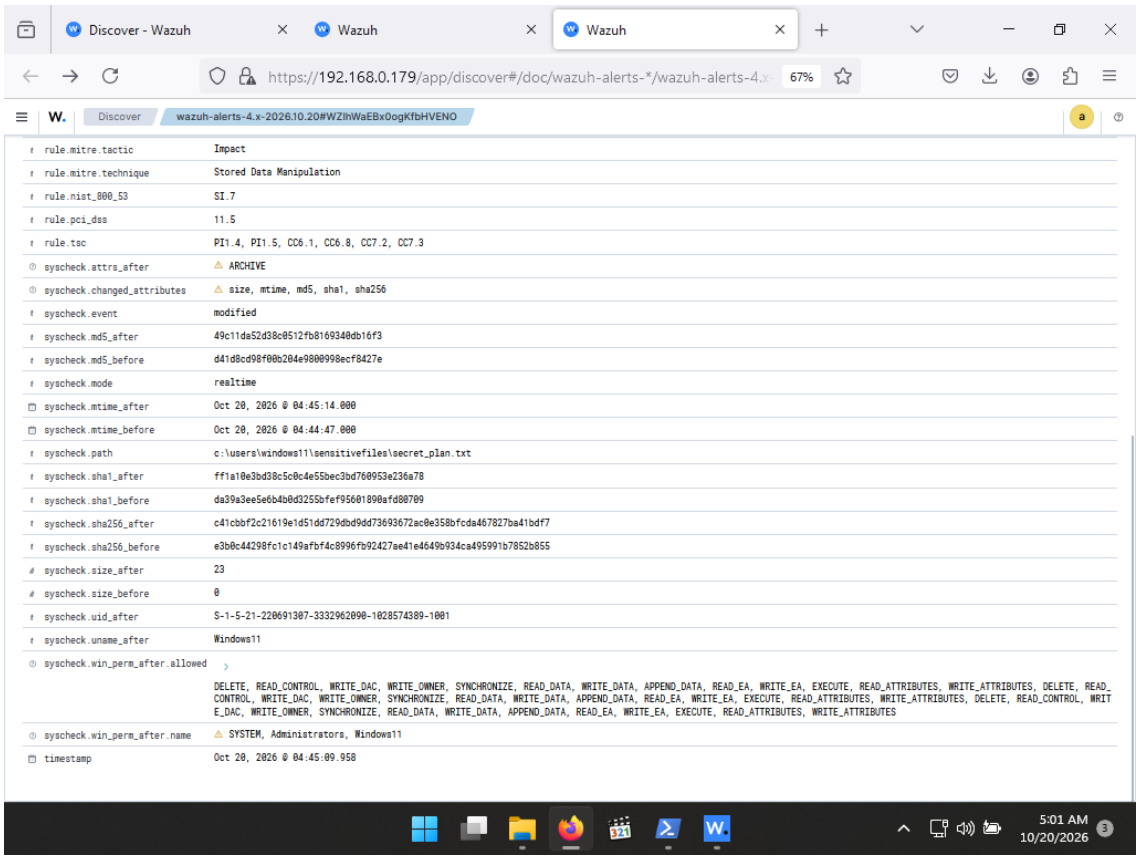



***Figure 18:*** *Document details (continued): syscheck.* fields showing the before and after values for size, mtime, MD5, SHA-1 and SHA-256.*

The details that show exactly what changed are in the syscheck fields:

| Attribute | Before | After |
|---|---|---|
| syscheck.changed_attributes | `size, mtime, md5, sha1, sha256` | `(same list: these are` |
| Size (bytes) | `0` | `23` |
| Modification time | `Oct 20, 2026 @ 04:44:47` | `Oct 20, 2026 @ 04:45:` |
| MD5 | `d41d8cd98f00b204e9800998ecf8427e` | `49c11da52d38c0512fb81` |
| SHA-1 | `da39a3ee5e6b4b0d3255bfef95601890afd80709` | `ff1a10e3bd38c5c0c4e55` |
| SHA-256 | `e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855` | `c41cbbf2c21619e1d51dd` |

**Interpretation.** The change list (size, mtime, md5, sha1, sha256) proves the file content changed, not just its metadata. The size grew from 0 to 23 bytes, and all three hashes changed. The "before" hashes are the well-known values of an empty file, which confirms the file was empty when it was created and that the 23 bytes were written afterwards. The modification time moved forward, confirming a write took place.

**Context fields.** The alert also records who and how: the file is in realtime mode, owned by user Windows11 (SID S-1-5-21-220691307-3332962090-1028574389-1001), carries the ARCHIVE attribute,

and its permissions list SYSTEM, Administrators and Windows11 with full access. The rule maps to MITRE ATT&CK T1565.001 (Stored Data Manipulation, tactic Impact) and to the compliance controls PCI DSS 11.5, HIPAA 164.312.c.1 and c.2, GDPR II_5.1.f, NIST 800-53 SI.7, GPG13 4.11 and several TSC criteria.

**Analyst observation.** The new modification time (04:45:14) is a few seconds later than the alert timestamp (04:45:09.958). The alert timestamp is set by the manager and the file time comes from the endpoint, so this most likely reflects a small clock offset between the two virtual machines. In production, time synchronization (NTP) matters when correlating events across hosts.

**Limitation.** The alert shows that the content changed and gives the new size and hash, but not what text was added. FIM tells the analyst that a file changed, when, by which account context and by how much; the content itself must be examined on the endpoint (or with the optional report_changes feature for text files).

## Question 2: False Positive Identification

**Task:** In a corporate environment, which folders would generate too many FIM alerts to be useful? Name two and explain why.

- **Temporary and cache folders** (for example C:\Windows\Temp, each user's AppData\Local\Temp and browser cache folders). Applications, installers and browsers create, rewrite and delete files in these locations constantly and legitimately. A real-time watch would produce hundreds or thousands of added, modified and deleted alerts per hour per endpoint, burying the few that matter and driving up indexer storage and analyst fatigue.
- **System log and update folders** (for example C:\Windows\Logs, application log directories, and C:\Windows\SoftwareDistribution used by Windows Update). Log files change on every write by design, and the update cache changes whenever patches are downloaded. Their hashes change all the time, so "integrity checksum changed" is the normal state rather than a sign of compromise.

Good practice is to monitor folders that should rarely change (system binaries, startup locations, configuration files, sensitive document shares) and to use the syscheck ignore option or custom rules to suppress volatile paths and file types. A well-tuned FIM policy keeps every alert meaningful.

## Question 3: Proactive Defense

**Task:** How could Wazuh's FIM capability be used to monitor for a specific ransomware behavior?

Ransomware leaves very visible file-system traces: it rewrites many files in a short time, renames them with a new extension, and drops ransom notes in many folders. FIM can detect each of these behaviors:

- **Mass modification.** Place realtime monitoring on user document folders and shares, then use a custom rule that raises severity when rule 550 (integrity checksum changed) fires many times within a short window. A user editing one file is normal; fifty files changing in thirty seconds is not.
- **Suspicious new extensions.** Alert when files are added with extensions commonly used by ransomware (for example .locked, .encrypted or .crypt), since encrypted copies usually appear as new or renamed files.

- **Ransom notes.** Alert on the creation of files with names such as README or HOW_TO_DECRYPT in monitored directories.
- **Canary files.** Plant decoy documents in monitored folders. Any modification or deletion of a file no user has a reason to touch is a high-confidence alert, exactly like the secret_plan.txt test in this lab.
- **Response.** Wazuh active response can be attached to the high-severity rule to isolate the host or kill the offending process while analysts investigate.

A sketch of the custom rules, which would be added to local_rules.xml on the manager (proposed design, not deployed in this lab):

```
<group name="syscheck,ransomware,">
  <!-- Many modified files in a short time -->
  <rule id="100100" level="12" frequency="20" timeframe="30">
    <if_matched_sid>550</if_matched_sid>
    <description>Possible ransomware: mass file modification detected.</description>
    <mitre><id>T1486</id></mitre>
  </rule>
  <!-- Files created with a ransomware-style extension -->
  <rule id="100101" level="12">
    <if_sid>554</if_sid>
    <field name="file" type="pcre2">(?i)\.(locked|encrypted|crypt)$</field>
    <description>Possible ransomware: file added with encrypted-file
extension.</description>
    <mitre><id>T1486</id></mitre>
  </rule>
</group>
```

FIM is a detective control: it flags the encryption as it happens, but it cannot stop it on its own. It should be combined with active response, backups and endpoint protection.

## Question 4: Tool Comparison

**Task:** How does a centralized tool like Wazuh provide an advantage over relying solely on built-in Windows features such as Event Logs?

| Aspect | Windows Event Logs alone | Centralized Wazuh FIM |
|---|---|---|
| Setup | File auditing must be enabled in policy and a SACL set on every folder, then the Security log read manually. | One directories line in ossec.conf, or a central agent group configuration. |
| Detail | Records that an access happened (for example event 4663) but not the file hash or the before and after values. | Records size, permissions, owner, and MD5, SHA-1 and SHA-256 before and after, as shown in Question 1. |
| Tamper resistance | Logs stay on the endpoint, where an attacker with admin rights can clear them. | Alerts are forwarded to the manager in near real time and stored off the host. |
| Scale and correlation | Reviewing each machine separately; no correlation across hosts. | One console for all agents, with searching, filtering and cross-host correlation. |

| Aspect | Windows Event Logs alone | Centralized Wazuh FIM |
|---|---|---|
| Context | Raw events with no classification. | Rules map alerts to severity levels, MITRE ATT&CK and PCI DSS, HIPAA, GDPR and NIST controls. |
| Response | Manual. | Custom rules, notifications and active response. |

In short, Windows Event Logs are a useful raw data source, but Wazuh adds integrity evidence, central storage, correlation and alerting. The two are complementary: Wazuh can also collect Windows Event Channel logs, so both can feed the same SOC console.

## 6. Lab Shutdown

At the end of the lab the agent and the Wazuh services were stopped cleanly. The agent was stopped from the Wazuh Agent Manager, and on the server the dashboard, indexer and manager services were stopped with systemctl and then verified with systemctl status.

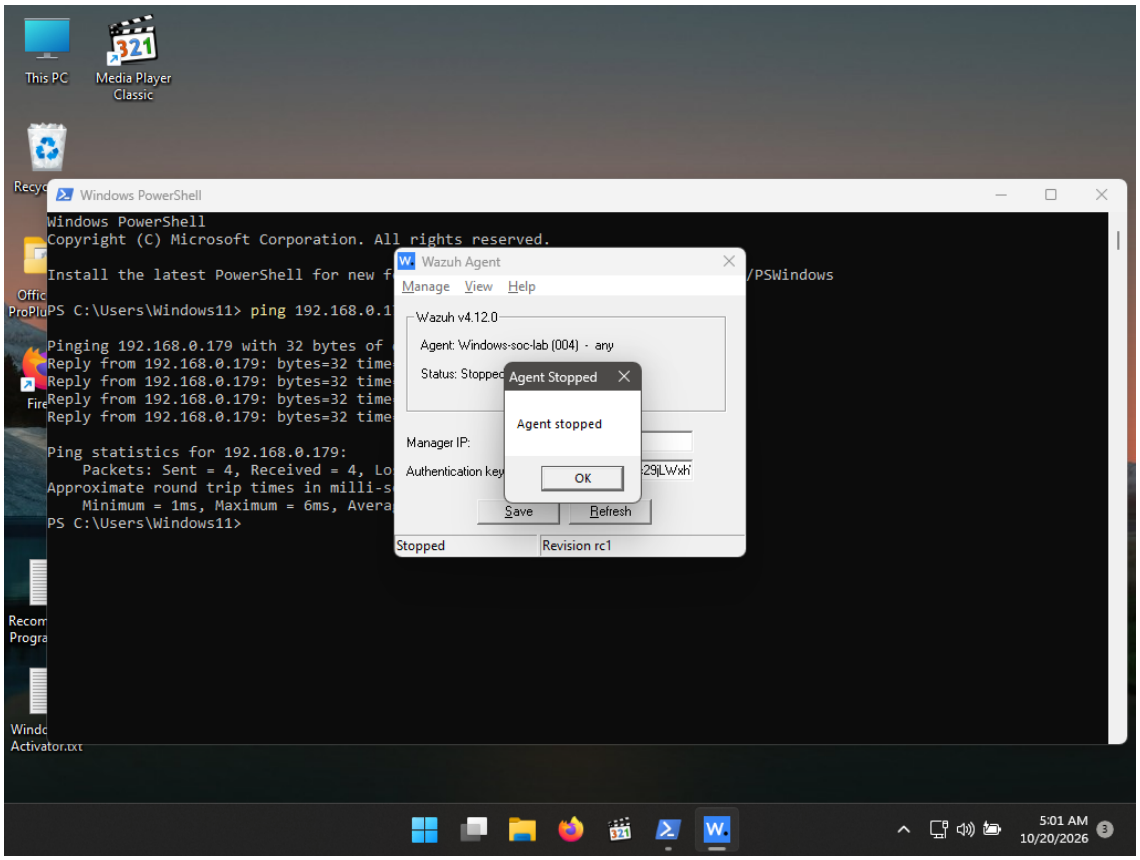



***Figure 19:*** *Wazuh Agent stopped at the end of the lab (October 20, 2026, 5:01 AM); the manager was still reachable by ping at 192.168.0.179.*

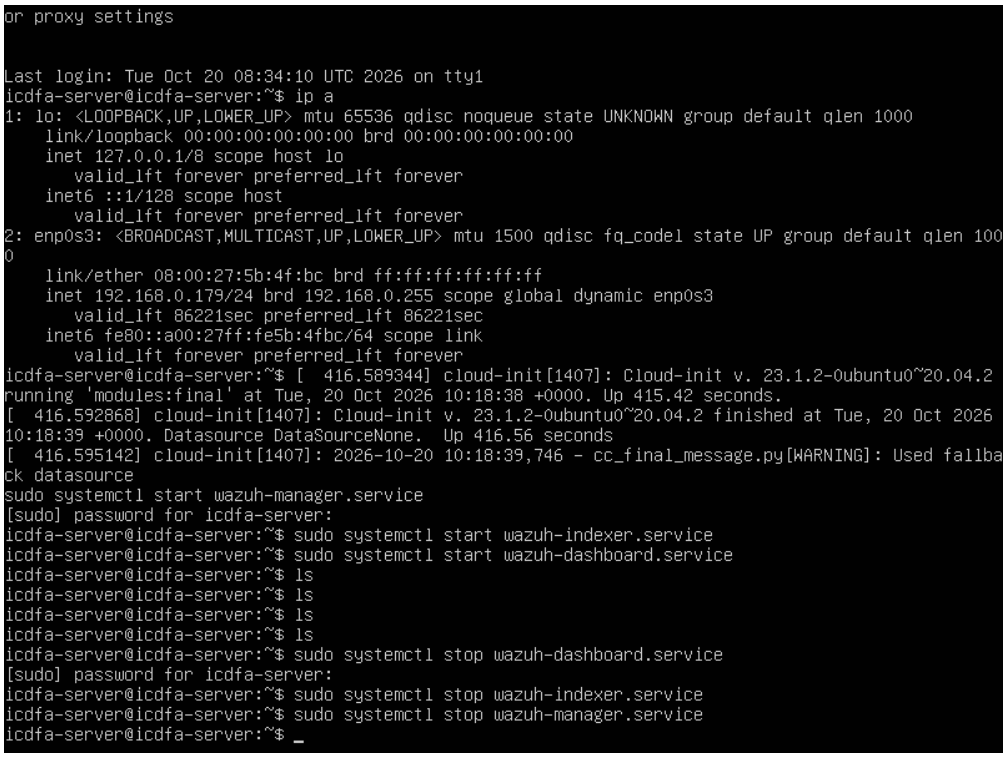



***Figure 20:*** *Server: wazuh-dashboard, wazuh-indexer and wazuh-manager stopped with systemctl stop (ip a shows 192.168.0.179).*

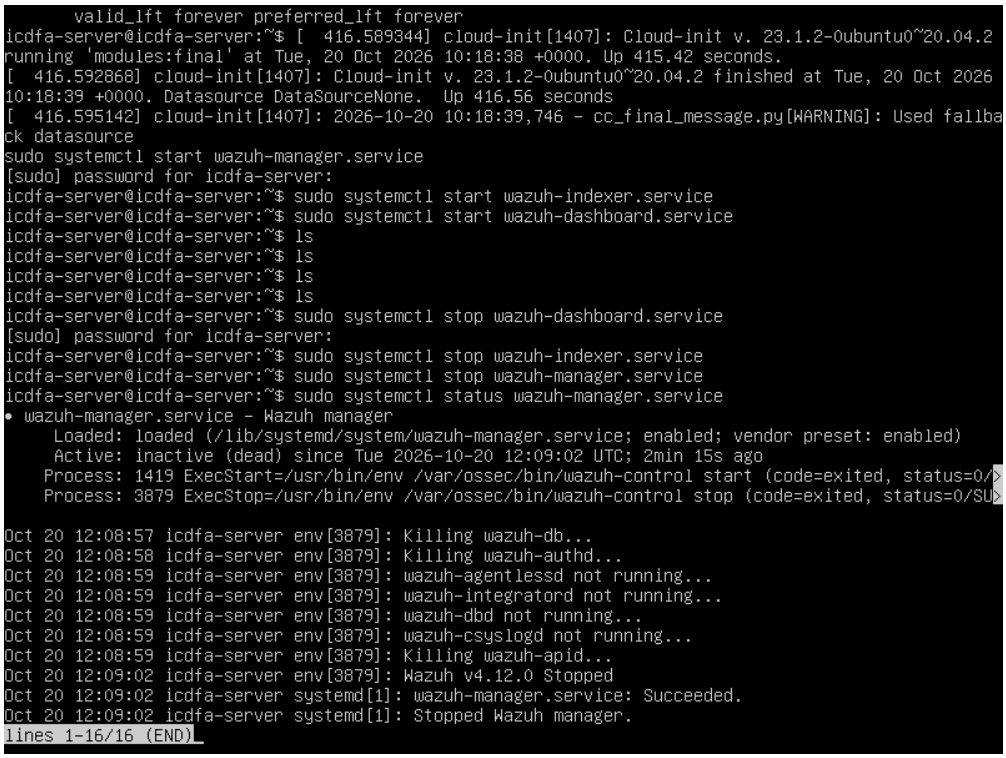



***Figure 21:*** *Status check: wazuh-manager is inactive (dead) after being stopped.*

```
icdfa-server@icdfa-server:~$ sudo systemctl stop wazuh-manager.service
icdfa-server@icdfa-server:~$ sudo systemctl status wazuh-manager.service
• wazuh-manager.service - Wazuh manager
     Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; vendor preset: enabled)
     Active: inactive (dead) since Tue 2026-10-20 12:09:02 UTC; 2min 15s ago
    Process: 1419 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/>
    Process: 3879 ExecStop=/usr/bin/env /var/ossec/bin/wazuh-control stop (code=exited, status=0/SU>

Oct 20 12:08:57 icdfa-server env[3879]: Killing wazuh-db...
Oct 20 12:08:58 icdfa-server env[3879]: Killing wazuh-authd...
Oct 20 12:08:59 icdfa-server env[3879]: wazuh-agentlessd not running...
Oct 20 12:08:59 icdfa-server env[3879]: wazuh-integratord not running...
Oct 20 12:08:59 icdfa-server env[3879]: wazuh-dbd not running...
Oct 20 12:08:59 icdfa-server env[3879]: wazuh-csyslogd not running...
Oct 20 12:08:59 icdfa-server env[3879]: Killing wazuh-apid...
Oct 20 12:09:02 icdfa-server env[3879]: Wazuh v4.12.0 Stopped
Oct 20 12:09:02 icdfa-server systemd[1]: wazuh-manager.service: Succeeded.
Oct 20 12:09:02 icdfa-server systemd[1]: Stopped Wazuh manager.
icdfa-server@icdfa-server:~$ sudo systemctl status wazuh-indexer.service
• wazuh-indexer.service - wazuh-indexer
     Loaded: loaded (/lib/systemd/system/wazuh-indexer.service; enabled; vendor preset: enabled)
     Active: inactive (dead) since Tue 2026-10-20 12:07:25 UTC; 4min 19s ago
       Docs: https://documentation.wazuh.com
    Process: 3259 ExecStart=/usr/share/wazuh-indexer/bin/systemd-entrypoint -p ${PID_DIR}/wazuh-ind>
   Main PID: 3259 (code=exited, status=143)

Oct 20 10:27:38 icdfa-server systemd-entrypoint[3259]: Oct 20, 2026 10:27:38 AM sun.util.locale.pro>
Oct 20 10:27:38 icdfa-server systemd-entrypoint[3259]: WARNING: COMPAT locale provider will be remo>
Oct 20 10:27:42 icdfa-server systemd-entrypoint[3259]: WARNING: A terminally deprecated method in j>
Oct 20 10:27:42 icdfa-server systemd-entrypoint[3259]: WARNING: System::setSecurityManager has been>
Oct 20 10:27:42 icdfa-server systemd-entrypoint[3259]: WARNING: Please consider reporting this to t>
Oct 20 10:27:42 icdfa-server systemd-entrypoint[3259]: WARNING: System::setSecurityManager will be >
Oct 20 10:30:11 icdfa-server systemd[1]: Started wazuh-indexer.
Oct 20 12:07:14 icdfa-server systemd[1]: Stopping wazuh-indexer...
Oct 20 12:07:25 icdfa-server systemd[1]: wazuh-indexer.service: Succeeded.
Oct 20 12:07:25 icdfa-server systemd[1]: Stopped wazuh-indexer.
lines 1-17/17 (END)
```

***Figure 22:*** *Status check: wazuh-manager and wazuh-indexer are inactive (dead) after being stopped.*

```
Oct 20 12:09:02 icdfa-server systemd[1]: Stopped Wazuh manager.
icdfa-server@icdfa-server:~$ sudo systemctl status wazuh-indexer.service
• wazuh-indexer.service - wazuh-indexer
     Loaded: loaded (/lib/systemd/system/wazuh-indexer.service; enabled; vendor preset: enabled)
     Active: inactive (dead) since Tue 2026-10-20 12:07:25 UTC; 4min 19s ago
       Docs: https://documentation.wazuh.com
    Process: 3259 ExecStart=/usr/share/wazuh-indexer/bin/systemd-entrypoint -p ${PID_DIR}/wazuh-ind>
   Main PID: 3259 (code=exited, status=143)

Oct 20 10:27:38 icdfa-server systemd-entrypoint[3259]: Oct 20, 2026 10:27:38 AM sun.util.locale.pro>
Oct 20 10:27:38 icdfa-server systemd-entrypoint[3259]: WARNING: COMPAT locale provider will be remo>
Oct 20 10:27:42 icdfa-server systemd-entrypoint[3259]: WARNING: A terminally deprecated method in j>
Oct 20 10:27:42 icdfa-server systemd-entrypoint[3259]: WARNING: System::setSecurityManager has been>
Oct 20 10:27:42 icdfa-server systemd-entrypoint[3259]: WARNING: Please consider reporting this to t>
Oct 20 10:27:42 icdfa-server systemd-entrypoint[3259]: WARNING: System::setSecurityManager will be >
Oct 20 10:30:11 icdfa-server systemd[1]: Started wazuh-indexer.
Oct 20 12:07:14 icdfa-server systemd[1]: Stopping wazuh-indexer...
Oct 20 12:07:25 icdfa-server systemd[1]: wazuh-indexer.service: Succeeded.
Oct 20 12:07:25 icdfa-server systemd[1]: Stopped wazuh-indexer.
icdfa-server@icdfa-server:~$ sudo systemctl status wazuh-dashboard.service
• wazuh-dashboard.service - wazuh-dashboard
     Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; vendor preset: enabled)
     Active: inactive (dead) since Tue 2026-10-20 12:03:36 UTC; 8min ago
    Process: 759 ExecStart=/usr/share/wazuh-dashboard/bin/opensearch-dashboards (code=exited, statu>
   Main PID: 759 (code=exited, status=0/SUCCESS)

Oct 20 11:59:40 icdfa-server opensearch-dashboards[759]: {"type":"response","@timestamp":"2026-10-2>
Oct 20 11:59:40 icdfa-server opensearch-dashboards[759]: {"type":"response","@timestamp":"2026-10-2>
Oct 20 11:59:42 icdfa-server opensearch-dashboards[759]: {"type":"response","@timestamp":"2026-10-2>
Oct 20 11:59:43 icdfa-server opensearch-dashboards[759]: {"type":"response","@timestamp":"2026-10-2>
Oct 20 12:00:01 icdfa-server opensearch-dashboards[759]: {"type":"log","@timestamp":"2026-10-20T12:>
Oct 20 12:00:03 icdfa-server opensearch-dashboards[759]: {"type":"log","@timestamp":"2026-10-20T12:>
Oct 20 12:03:14 icdfa-server systemd[1]: Stopping wazuh-dashboard...
Oct 20 12:03:20 icdfa-server opensearch-dashboards[759]: {"type":"log","@timestamp":"2026-10-20T12:>
Oct 20 12:03:36 icdfa-server systemd[1]: wazuh-dashboard.service: Succeeded.
Oct 20 12:03:36 icdfa-server systemd[1]: Stopped wazuh-dashboard.
lines 1-16/16 (END)
```

***Figure 23:*** *Status check: wazuh-indexer and wazuh-dashboard are inactive (dead).*

# 7. Conclusion

The lab demonstrated a working FIM pipeline from endpoint to SOC console. After the agent was registered and the SensitiveFiles folder was set to real-time monitoring, every create, rename, modify and delete action appeared in Wazuh within seconds as a syscheck alert, including before and after checksums, size, timestamps, ownership and MITRE ATT&CK context. Analysing the modified-file alert showed how the hash and size fields prove a content change, and the analysis questions showed that FIM must be tuned to avoid noisy folders, can be extended with custom rules to catch ransomware behavior, and gives a central, tamper-resistant and richer view than Windows Event Logs alone.